APPROXIMATION ALGORITHMS

- VERTEX-COVER PROBLEM
- TRAVELING-SALESMAN PROBLEM

Approximation Algorithms: Overview

Many problems of practical significance are NP-complete, yet they are too important to abandon merely because we don't know how to find an optimal solution in polynomial time.

There are at least three ways to get around NP-completeness.

- **First**, if the actual inputs are small, an algorithm with exponential running time may be perfectly satisfactory.
- **Second**, we may be able to isolate important special cases that we can solve in polynomial time.
- **Third**, we might come up with approaches to find close-optimal solutions in polynomial time (either in the worst case or the several expected cases) using approximation algorithms.

Approximation Algorithms: Overview

In practice, close-optimality is often good enough. We call an approximation algorithm that returns close-optimal solutions. We shall see few examples of polynomial-time approximation algorithms for NP-complete problems.

- **Definition:** An algorithm that returns close optimal solutions in polynomial time is known as approximation algorithm.

Approximation Ratio

An approximate algorithm has an approximation ratio of $\rho(n)$ if for any input of size n, $\max(C/C^*, C^*/C) \leq \rho(n)$

where C = cost of solution obtain by approximation algorithm

C^* = cost of optimal solution.

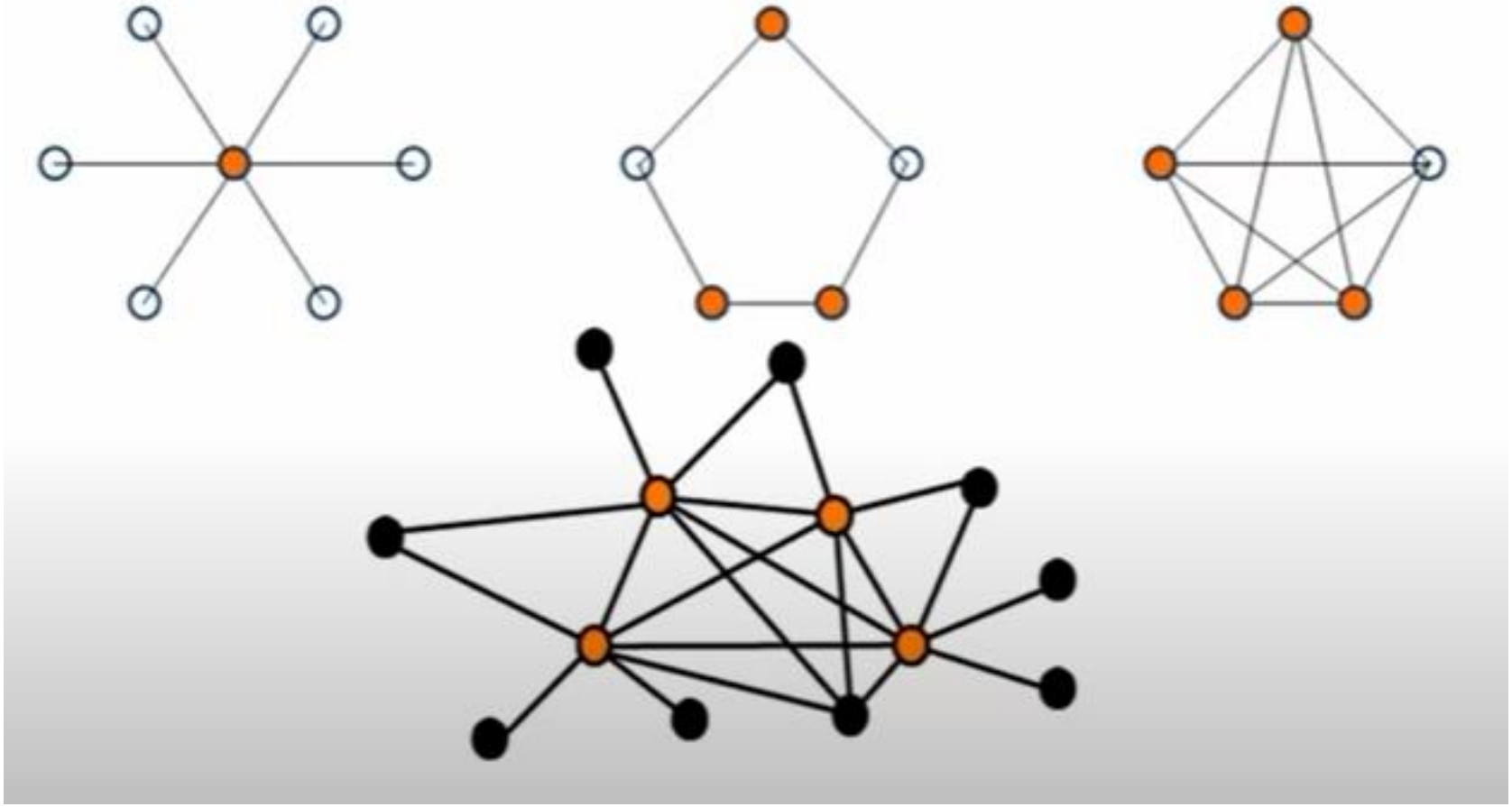
- This definition applies to both minimization and maximization problems. Also called $\rho(n)$- approximation algorithm.
- For a maximization problem, $0 < C \leq C^*$ and the ratio C^*/C gives the factor by which the cost of an optimal solution is larger than the cost of the approximate solution.
- Similarly, for a minimization problem, $0 < C^* \leq C$, and the ratio C/C^* gives the factor by which the cost of the approximate solution is larger than the cost of an optimal solution.

(*We assume that all solutions have positive cost and these ratios are always well defined.*)

1. *Vertex-Cover problem*

- In graph theory, a vertex cover of a graph is a set of vertices that includes at least one endpoint of every edge of the graph.
- In other words, finding the minimum size vertex set that covers all edge of a given graph G.
- In computer science, the problem of finding a minimum vertex cover is a classical optimization problem. It is NP-hard, so it cannot be solved by a polynomial-time algorithm if P ≠ NP.
- Vertex cover of an undirected graph G=(V,E) is a subset V'≤ V such that ∀(u,v) ∈ edge of G, then either u ∈ V' or v ∈ V'. The size of a vertex cover is the number of vertices in it.
- We call such a vertex cover an *optimal vertex cover*. This problem is the optimization version of an NP-complete decision problem.
- Even though we don't know till date how to find an optimal vertex cover in a graph G in polynomial time, we can efficiently find vertex cover that is close-optimal sol^{n} using approximation.

1. *Vertex-Cover problem*



Vertex Cover: Approximation Algorithm

- The following approximation algorithm takes as input an undirected graph G and returns a vertex cover whose size is guaranteed to be no more than twice the size of an optimal vertex cover.

Pseudocode

```
Approx-Vertex-Cover (G)
1.  C := Ø;
2.  while E ≠ Ø do
3.        select any (u,v) in E which was not selected before;
4.        C:= C ∪ {u,v};
5.        remove from E every edge incident on either u or v
6.  return C
```

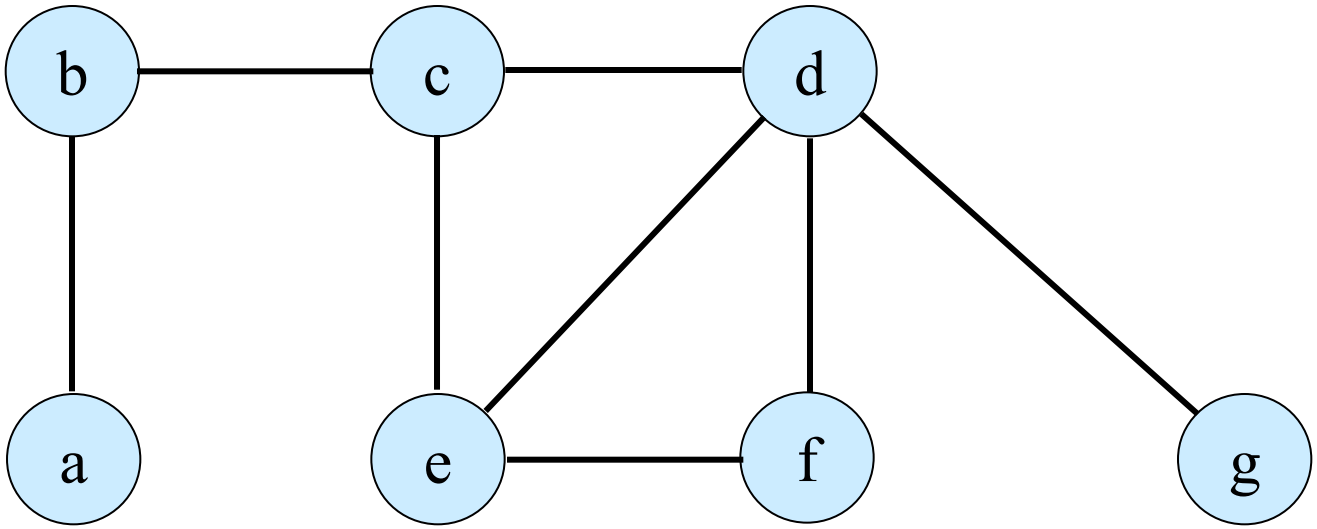
Vertex Cover: Approximation Algorithm Analysis

In the previous Algorithm, the variable C contains the vertex cover being constructed.

- Line 1 initializes C to the empty set.
- The loop of lines 2–5 repeatedly picks an edge (u,v) from E, adds its both endpoints u and v to C, and deletes all incident edges in E that are covered by either u or v.
- Finally, line 6 returns the vertex cover C.

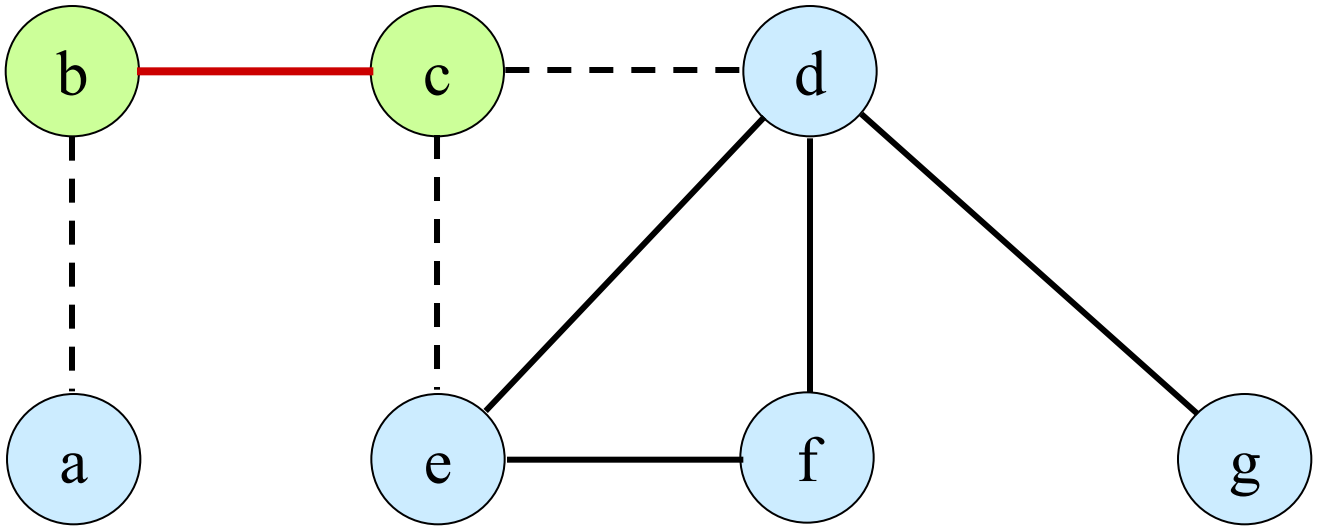
The running time of this algorithm is O (V+E)

EXAMPLE: APPROX. VERTEX COVER



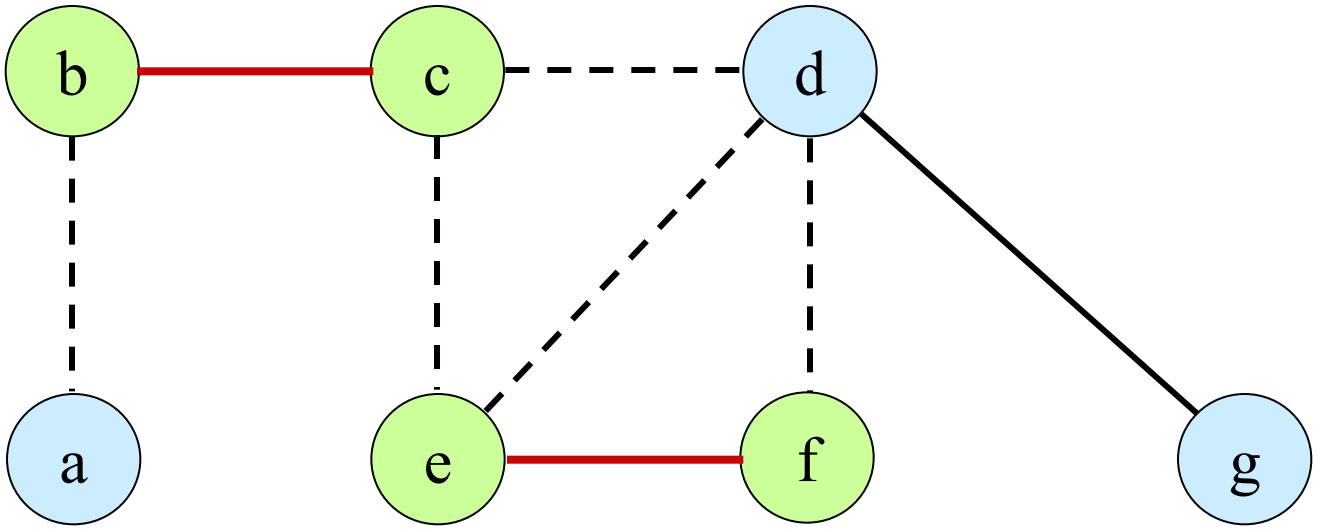
The **input graph G**, which has **7 vertices** and **8 edges**.

EXAMPLE: APPROX. VERTEX COVER



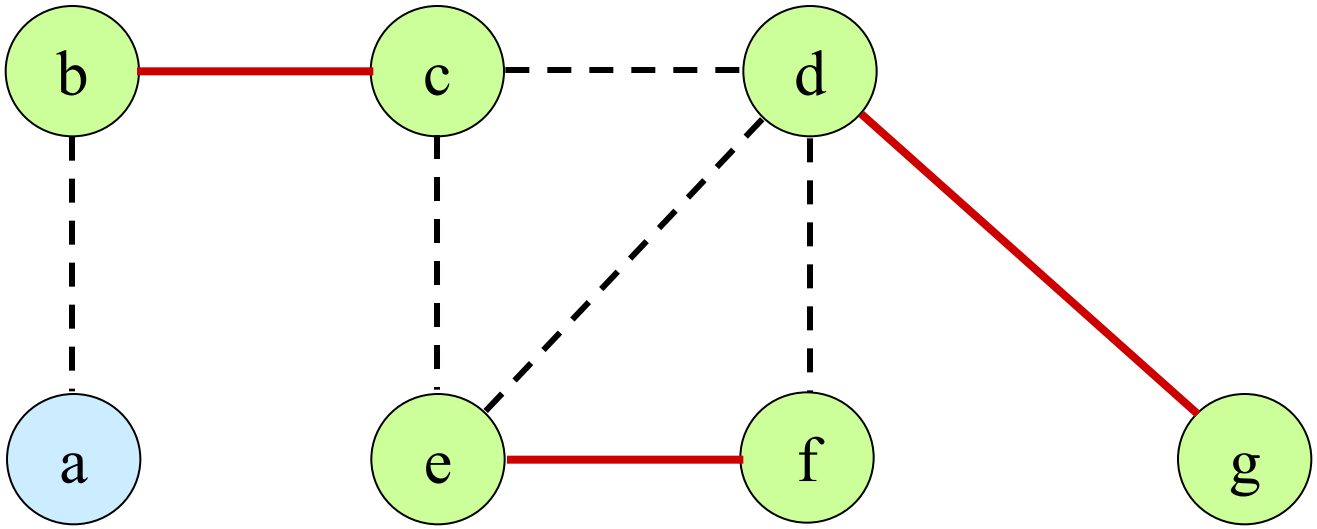
- The edge (b,c), shown **heavy**, is the first edge chosen by APPROX-VERTEX-COVER. Vertices b and c, shown **lightly shaded**, are added to the set C containing the vertex cover being created.
- Edges (a,b), (c,e), and (c,d), shown **dashed, are removed** since they are now covered by some vertex in C.

EXAMPLE: APPROX. VERTEX COVER



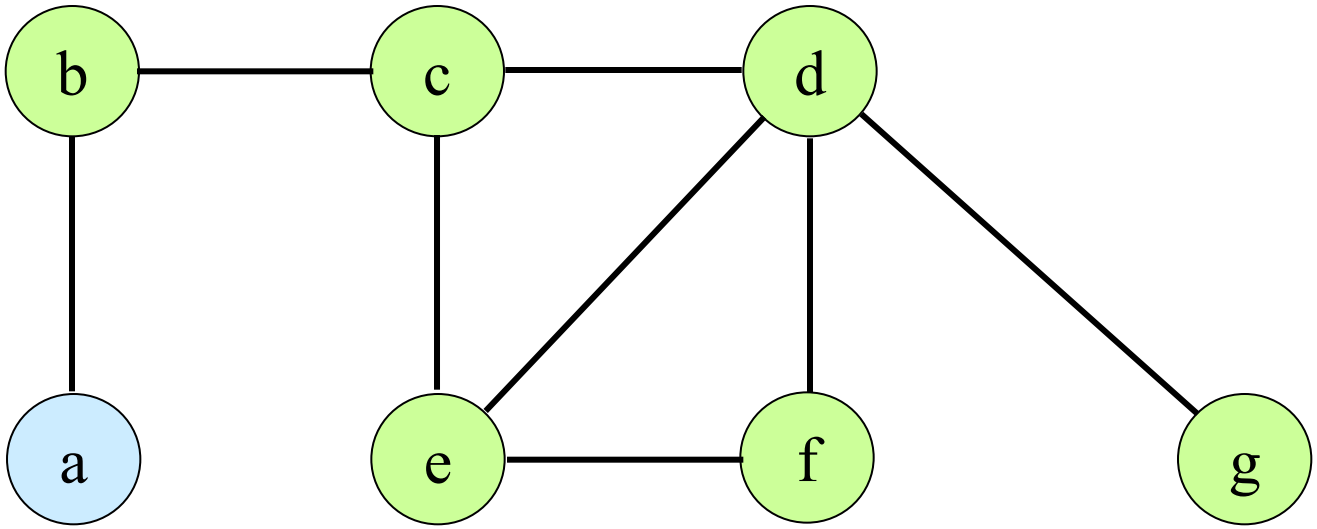
- **Edge (e, f) is chosen**; vertices e and f are added to C.

EXAMPLE: APPROX. VERTEX COVER



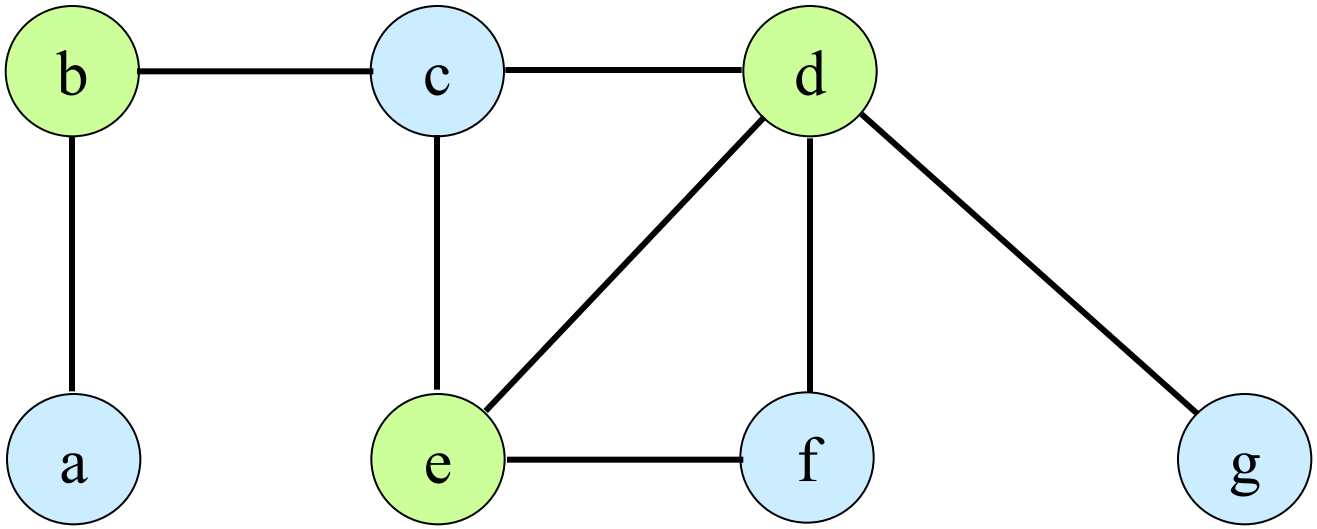
- **Edge (d, g) is chosen**; vertices d and g are added to C.

EXAMPLE: APPROX. VERTEX COVER



- The set C, which is the vertex cover produced by the approximation algorithm contains the six vertices {b, c, d, e, f, g }

EXAMPLE: APPROX. VERTEX COVER



While, the optimal vertex cover for this graph G contains only three vertices: b, d, and e.

EXAMPLE: APPROX. VERTEX COVER

Approximate Vertex Cover algorithm is a 2-approximation algorithm or has an approximation ratio bound of 2.

Proof: **The set C of vertices that is returned by above algorithm is a vertex cover, since the algorithm loops until every edge in E[G] has been covered by some vertex in C.**

- **Let A = set of edges selected.**
- **Note that no two edges in A share a common endpoint which implies $|C| = 2|A|$.**
- **Also implies $|A| \leq |C^*|$, because optimal cover C^* must include at least one endpoint of each edge in A.**
- **Hence, $|C| \leq 2|C^*|$ or $\frac{C}{C^*} \leq 2$**

The traveling-salesman problem

- we are given a complete undirected graph G=(V;E) that has a nonnegative integer cost c(u,v) associated with each edge (u,v) ∈ E, and we must find a Hamiltonian cycle (a tour) of G with minimum cost.
- let c(A) denote the total cost of the edges in the subset A≤ E: **c(A) =** $\sum_{(u,v)\in A} c(u,v)$.
- Given complete, undirected graph. For all vertices, u,v,w ∈ V require: c(u,w) ≤ c(u,v) + c(v,w).
- **Note:** Holds if C denotes distance on a plane.
- With triangle inequality, TSP is still NP-complete.

Traveling-salesman problem with the triangle inequality

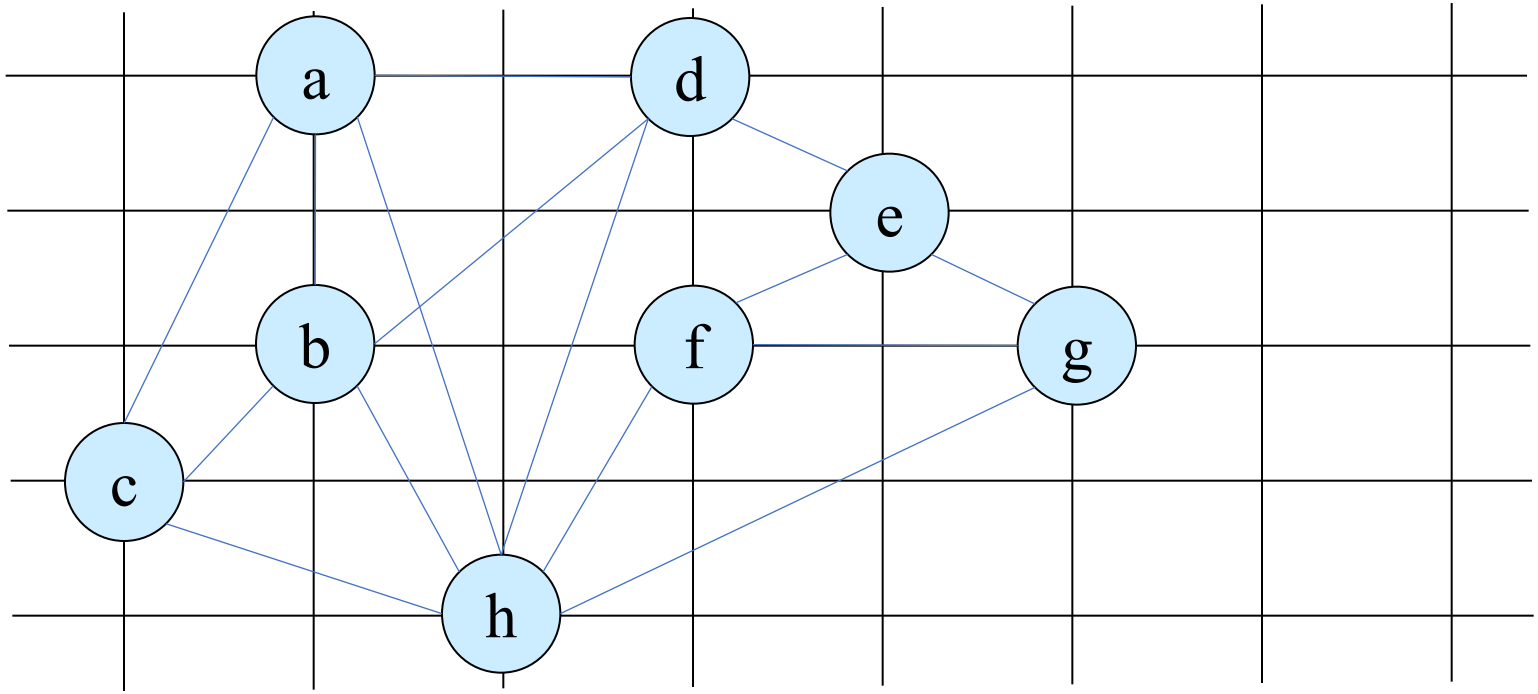
- **The following algorithm computes a near-optimal tour of an undirected graph G using minimum-spanning-tree algorithm (MST-PRIM).**
- **We shall then use the minimum spanning tree to create a tour whose cost is no more than twice that of the minimum spanning tree's weight, as long as the cost function satisfies the triangle inequality**.

Approx- TSP-Tour(G,c)

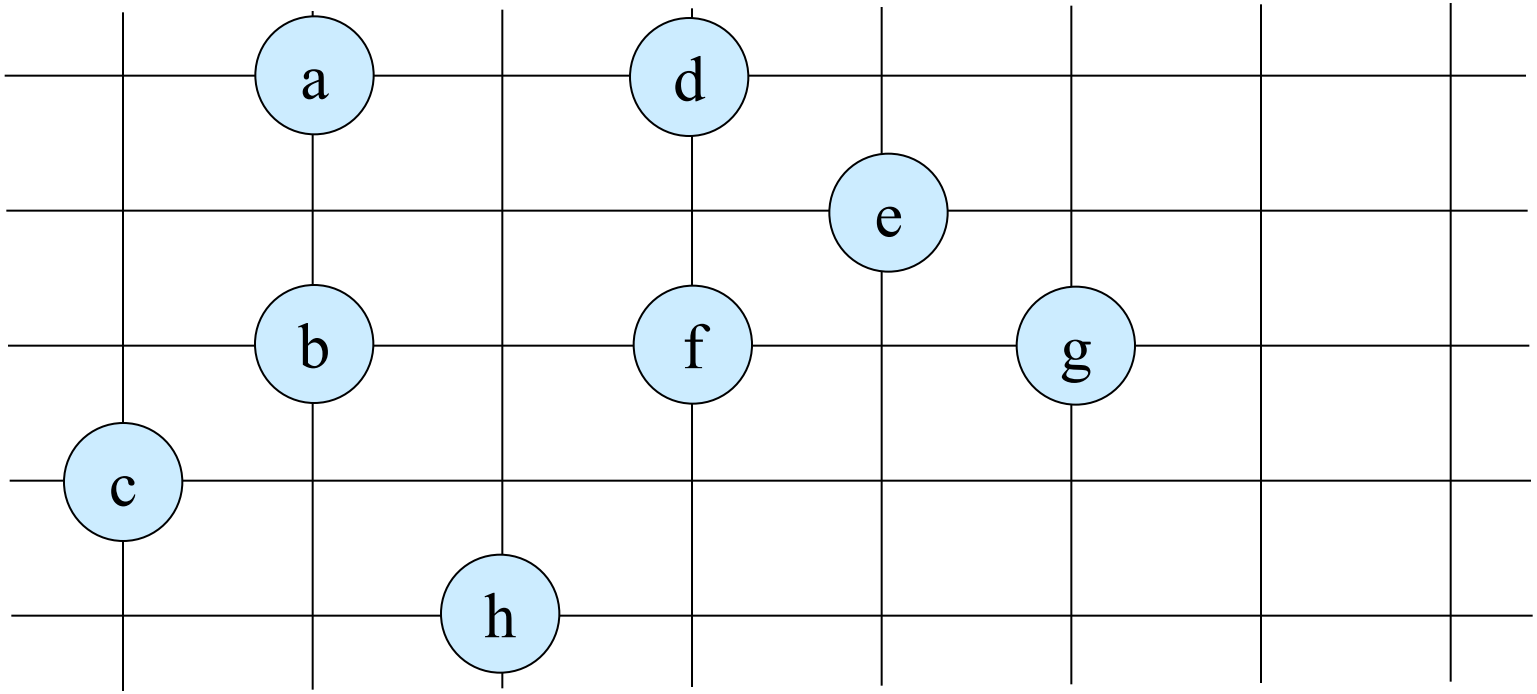
1. select a vertex r ∈ v[G] to be a "root" vertex;
2. call MST-Prim(G,c,r) to construct MST with root r;
3. let L = vertices on preorder tree walk of MST;
4. let H = cycle that visits vertices in the order L and finally visit the staring vertex;

return H

TSP Approximation Algorithm

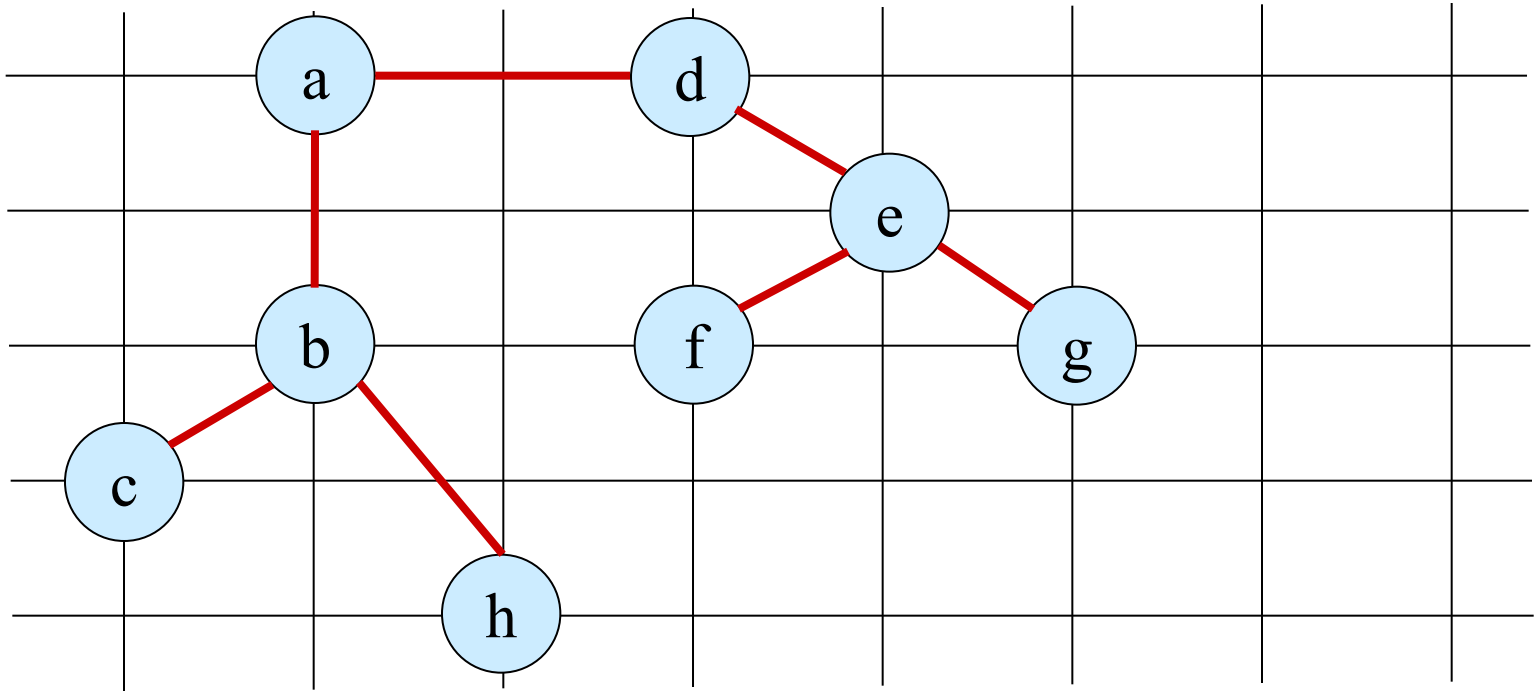


TSP Approximation Algorithm



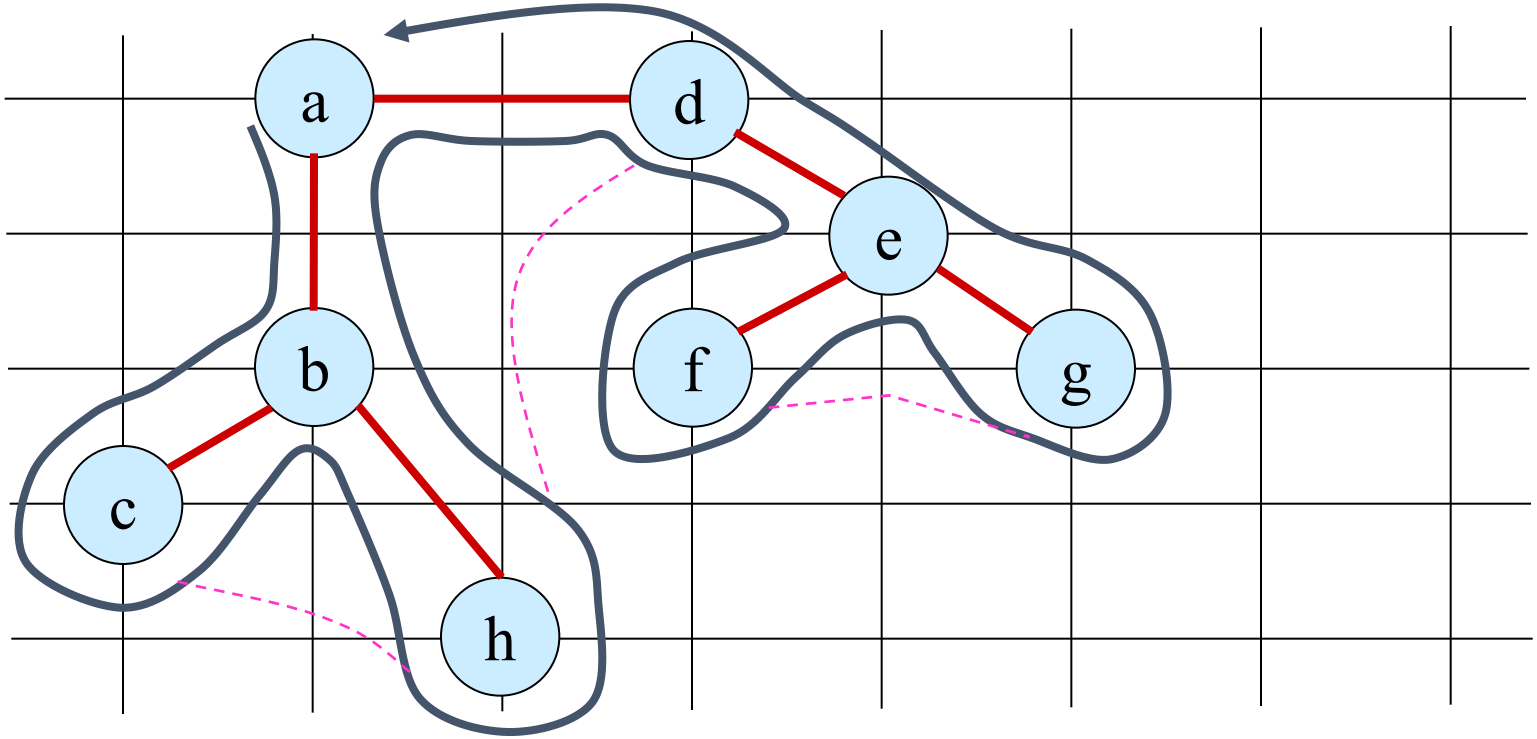
Let us consider a complete undirected graph. Vertices lie on intersections of integer grid lines. For example, ***f*** is one unit to the right and two units up from ***h***. The cost function between two points is the ordinary Euclidean distance.

TSP Approximation Algorithm



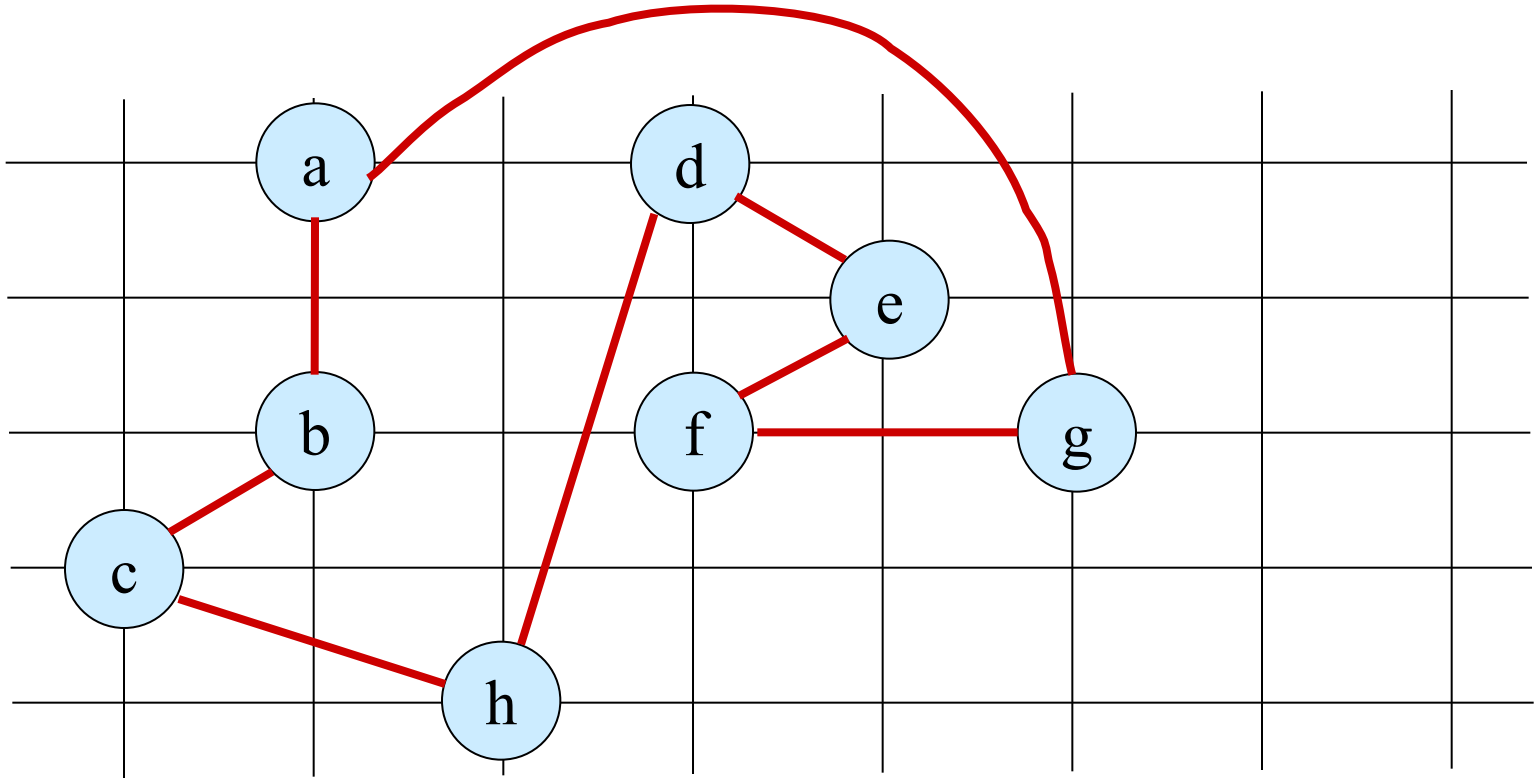
A minimum spanning tree T of the complete graph, as computed by MST-PRIM. Vertex ‘a’ is the root vertex. Only edges in the minimum spanning tree are shown. The vertices happen to be labeled in such a way that they are added to the main tree by MST-PRIM in alphabetical order.

TSP Approximation Algorithm



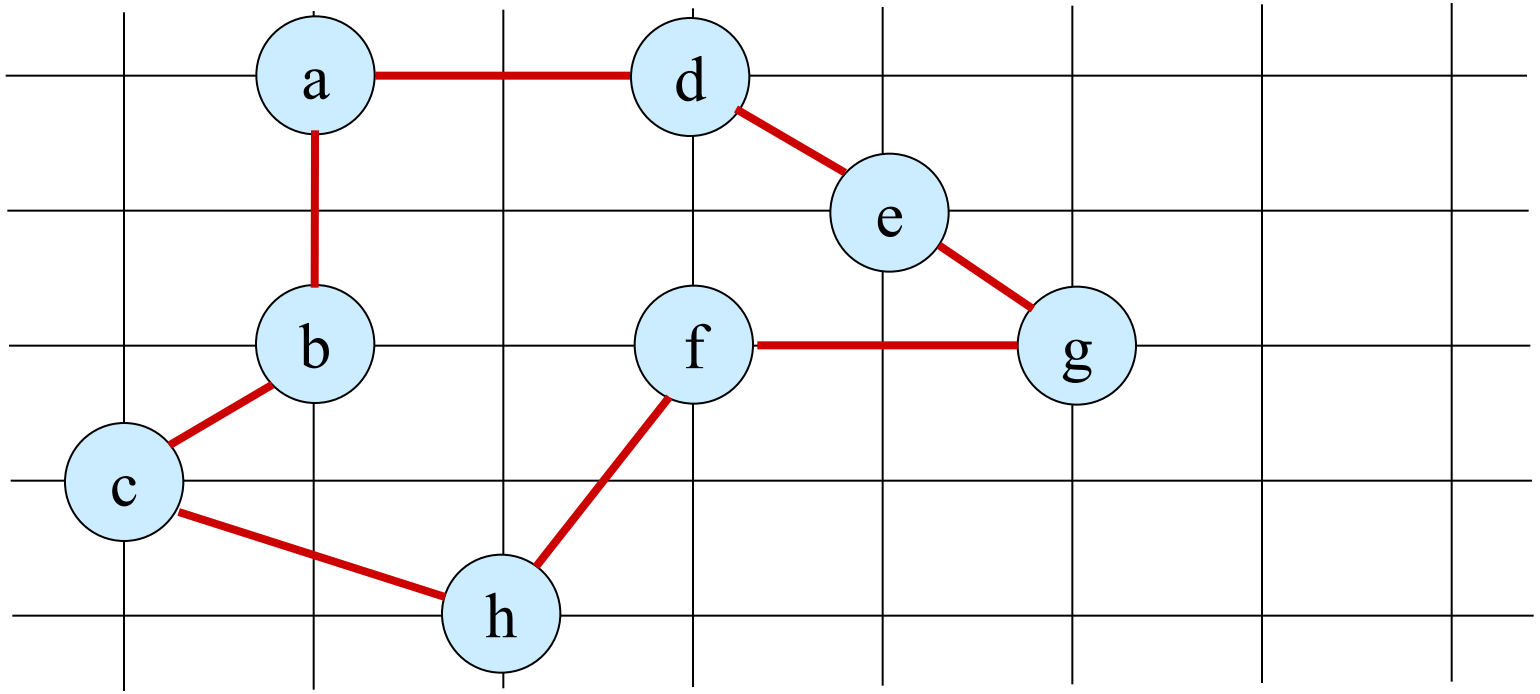
- A walk of T, starting at ‘a’. A full walk of the tree visits the vertices in the order a, b, c, b, h, b, a, d, e, f, e, g, e, d, a.
- A preorder walk of T lists a vertex just when it is first encountered, as indicated by the dot next to each vertex, yielding the ordering a, b, c, h, d, e, f, g.

TSP Approximation Algorithm



Computed Tour

Optimal TSP solution



Optimal Tour

APPROX-TSP-TOUR is a polynomial-time 2-approximation algorithm for the TSP with the triangle inequality.

Proof:

Let H^* = optimal tour, T = MST.
Deleting an edge of H^* yields a spanning tree.
So, $c(T) \le c(H^*)$.

Consider full walk W of T.
Ex: a,b,c,b,h,b,a,d,e,f,e,g,e,d,a.
Visits each edge twice
$\Rightarrow c(W) = 2c(T)$.

Hence, $c(W) \le 2c(H^*)$.

Triangle inequality $\Rightarrow$ can delete any vertex from W and cost doesn't increase.

Deleting all but first appearance of each vertex yields preorder walk.
Ex: a,b,c,h,d,e,f,g.

Corres. cycle H = W with some edges removed.
Ex: a,b,c,h,d,e,f,g,a.

$c(H) \le c(W)$.

Implies $c(H) \le 2c(H^*)$.